Delve Architectural Blueprint

A Comprehensive Guide to Recreating Every System

Delve Project

March 2026

Contents

1 Project Overview & Build System **2**
- 1.1 Prerequisites . . . 2
- 1.2 Build Commands & Targets . . . 2
- 1.3 Dependency Management . . . 2
- 1.4 Shader Compilation Pipeline . . . 2

2 Engine Layer Architecture **3**
- 2.1 Application Framework . . . 3
- 2.2 GPU Abstraction . . . 4
- 2.3 Camera System . . . 5
- 2.4 Input System . . . 5
- 2.5 Asset Manager . . . 5
- 2.6 Task System . . . 5
- 2.7 Watchdog . . . 6
- 2.8 glTF Loader . . . 6
- 2.9 ImGui Integration . . . 6
- 2.10 IPC Agent Server . . . 6
- 2.11 Background Renderer . . . 6
- 2.12 Frame Capture . . . 6

3 Terrain Pipeline **6**
- 3.1 Step 1: Noise Generation . . . 6
  - 3.1.1 Elevation Layer . . . 7
  - 3.1.2 River Mask . . . 7
  - 3.1.3 Worley (Cellular) Layer . . . 7
- 3.2 Step 2: Composition . . . 8
- 3.3 Step 3: Contour Detection . . . 8
  - 3.3.1 Marching Squares . . . 8
  - 3.3.2 Plateau Detection . . . 9
- 3.4 Step 4: Hex Column Generation . . . 9
  - 3.4.1 Axial Hex Coordinates . . . 9
  - 3.4.2 Column Generation (`basalt.cpp`) . . . 9
- 3.5 Step 5: Lava/Void Generation . . . 9
- 3.6 Step 6: Mesh Assembly . . . 10
- 3.7 Step 7: GPU Upload & Rendering . . . 10

4 Shader Architecture **11**

4.1 Shared Includes . . . 11
4.2 Terrain Shaders . . . 11
4.3 Lava Shaders . . . 11
4.4 Background Shaders . . . 11
4.5 Contour Shaders . . . 12
4.6 Compute Shaders . . . 12
4.7 Instanced Terrain . . . 12
4.8 PBR Static . . . 12

5 Procedural Animation System (Rig System) 12
5.1 Skeleton . . . 12
5.2 ECS System Pipeline . . . 13
5.2.1 1. PlayerMovementSystem . . . 13
5.2.2 2. ActorGroundingSystem . . . 13
5.2.3 3. GaitSystem — Distance-Based Procedural Stepping . . . 13
5.2.4 4. GrabDriveSystem . . . 14
5.2.5 5. IKSystem — Two-Bone Analytical IK . . . 14
5.2.6 6. OverlaySystem — Additive Overlays . . . 14
5.2.7 7. LookAtSystem — Head Tracking . . . 14
5.2.8 8. SkeletonFinaliseSystem . . . 15
5.2.9 9. AnimationLogSystem . . . 15
5.3 SmoothDamp — Critically Damped Spring . . . 15

6 Rig Renderer 15

7 glTF Asset Pipeline 16
7.1 Loading Pipeline . . . 16
7.2 Instanced Terrain Rendering . . . 16

8 ECS Architecture 17
8.1 Component Catalog . . . 17
8.2 System Registration Pattern . . . 17

9 Testing Framework 17
9.1 Test Harness . . . 17
9.2 Metric Suites . . . 18
9.3 Test Files . . . 18

10 Library Reference 18

11 Entry Point & Runtime Flow 20

1 Project Overview & Build System

Delve is a procedural hex-grid terrain generator written in C++20. It renders isometric 3D landscapes using layered Perlin/Worley noise with real-time parameter editing via ImGui. The GPU backend uses SDL3-GPU, which abstracts over Vulkan, Metal, and DirectX 12.

1.1 Prerequisites

- **SDL3** — system package (windowing, GPU abstraction)
- **glslc** — SPIR-V shader compiler (from Vulkan SDK or shaderc)
- **CMake** $\geq$ 3.20
- A C++20-capable compiler

1.2 Build Commands & Targets

```
cmake -B build -DCMAKE_BUILD_TYPE=Release    # configure
cmake --build build -j$(nproc)               # build all
./build/topogen                              # run application
cmake --build build --target delve_tests     # build tests
./build/delve_tests                          # run tests (JSON to stdout)
```

Table 1: CMake build targets

Target	Type	Description
`imgui`	Static lib	ImGui core + SDL3/SDL-GPU backends
`topo_engine`	Static lib	Reusable engine framework (18 source files)
`topogen`	Executable	Main application (21 game sources)
`delve_tests`	Executable	Headless test runner (`EXCLUDE_FROM_ALL`)
`delve_metrics`	Executable	Headless metrics binary (`EXCLUDE_FROM_ALL`)
`shaders`	Custom	GLSL → SPIR-V compilation

1.3 Dependency Management

Table 2: External dependencies

Library	Version	Method	CMake Target
Flecs	v4.0.5	FetchContent	`flecs::flecs_static`
nlohmann/json	v3.11.3	FetchContent	`nlohmann_json::nlohmann_json`
GLM	1.0.1	FetchContent	`glm::glm`
ImGui	latest	Git submodule	`imgui` (static lib)
cgltf	latest	Vendored header	`third_party/cgltf/`
stb_image	latest	Vendored header	`third_party/stb/`
FastNoiseLite	latest	Vendored header	`src/game/terrain/`

1.4 Shader Compilation Pipeline

Shaders are written in GLSL 4.5 in `src/shaders/` and compiled to SPIR-V via `glslc`. Three shared include files (`coord.glsl`, `lighting_common.glsl`, `pbr_common.glsl`) trigger a full rebuild of all dependents when modified.

```
# Graphics shader compilation (per stage)
glslc -fshader-stage=vertex -I src/shaders/ -o build/shaders/X.spv src/
    shaders/X.glsl
# Compute shader compilation
glslc -fshader-stage=compute -I src/shaders/ -o build/shaders/X.spv src/
    shaders/X.glsl
```

Compile definitions propagated to C++: `SHADER_DIR` (SPIR-V output path), `ASSET_DIR` (project root `assets/`), `GLM_FORCE_DEPTH_ZERO_TO_ONE`.

2 Engine Layer Architecture

The engine lives in `src/engine/` and provides a reusable framework independent of game logic.

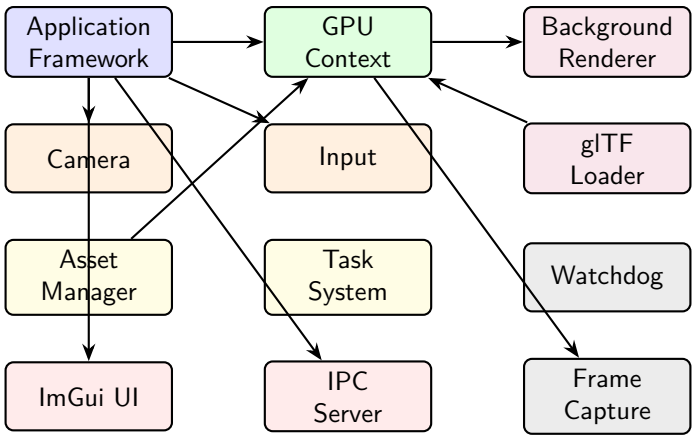

Figure 1: Engine module dependency graph

2.1 Application Framework

`app.h/cpp` defines the abstract `Application` base class with a fixed-timestep main loop.

Constants:

- `FIXED_DT = 1/60 s` (60 FPS fixed timestep)
- `MAX_FRAME_TIME = 0.25 s` (clamp to prevent spiral of death)

Lifecycle hooks (virtual methods overridden by the game):

- `on_init(GpuContext&, flecs::world&)`
- `on_event(SDL_Event&, flecs::world&)`
- `on_render_tool(GpuContext&, FrameContext&, flecs::world&)` — tool window (ImGui)
- `on_render_game(GpuContext&, FrameContext&, flecs::world&)` — game window (terrain)
- `on_cleanup(flecs::world&)`

Main loop pseudocode:

```
while (!quit) {
  poll_agent_server();          // IPC commands
  dt = clamp(elapsed, 0, MAX_FRAME_TIME);
```

```
  accumulator += dt;
  while (accumulator >= FIXED_DT) {
    ecs_world.progress(FIXED_DT);
    accumulator -= FIXED_DT;
  }
  poll_sdl_events();              // input, window events
  asset_manager.check_for_updates();  // shader hot-reload
  acquire_tool_frame();  on_render_tool();  end_frame();
  acquire_game_frame();  on_render_game();  end_frame();
}
```

How to Recreate

The dual-window setup renders ImGui controls in one window and the 3D viewport in another. Each window gets its own swapchain acquisition and render pass. The tool window is always rendered; the game window is conditional on `wants_game_window_open()`.

2.2 GPU Abstraction

`gpu/gpu.h/cpp` wraps SDL3-GPU with higher-level upload and frame management.

Key structures:

```
struct UploadManager {
  SDL_GPUTransferBuffer *buffer;    // persistent 8 MB staging buffer
  uint8_t *mapped;                  // persistently mapped CPU pointer
  uint32_t capacity;                // 8 * 1024 * 1024
  uint32_t cursor;                  // current write offset
  // alloc(): 256-byte aligned bump allocation
  // reset(): cursor = 0 at frame start
};

struct GpuContext {
  SDL_Window *window, *game_window;
  SDL_GPUDevice *device;
  UploadManager upload_manager;
};

struct FrameContext {
  SDL_GPUCommandBuffer *cmd;
  SDL_GPUTexture *swapchain;
  SDL_GPURenderPass *render_pass;
  uint32_t swapchain_w, swapchain_h;
};
```

Buffer utilities:

- `gpu_create_buffer()` — allocates GPU buffer with usage flags
- `gpu_upload_buffer()` — creates buffer and uploads data in one call
- `upload_rgba_texture()` — creates sRGB or linear texture with REPEAT sampler

How to Recreate

The UploadManager uses a single 8 MB transfer buffer with bump allocation (256-byte aligned). This avoids per-frame buffer creation overhead. Reset the cursor to zero at the start of each game frame, not the tool frame.

2.3 Camera System

The isometric camera in `camera/camera.h/cpp` uses a custom projection combining 2:1 tile aspect with height scaling.

Isometric constants:

$$\text{TW} = 2.0 \quad \text{(tile width scale, X axis)} \tag{1}$$
$$\text{TH} = 1.0 \quad \text{(tile height scale, Y axis)} \tag{2}$$
$$\text{HS} = 12.5 \quad \text{(height scale = ISO\_HEIGHT\_SCALE / HEX\_SIZE)} \tag{3}$$

Coordinate transform (world → isometric screen):

$$\text{iso}_x = (w_x - w_y) \cdot \text{TW} \tag{4}$$
$$\text{iso}_y = (w_x + w_y) \cdot \text{TH} - w_z \cdot \text{HS} \tag{5}$$
$$\text{iso}_z = (w_x + w_y) \cdot \text{TH} + w_z \quad \text{(depth)} \tag{6}$$

View matrix (column-major):

```
view = mat4(
  vec4( TW,  TH,  TH, 0),  // world X -> iso
  vec4(-TW,  TH,  TH, 0),  // world Y -> iso
  vec4(  0, -HS, 1.0, 0),  // world Z -> iso
  vec4(  0,   0,   0, 1)
);
```

Follow smoothing uses exponential decay: $t = 1 - e^{-\text{speed} \cdot dt}$, with default `follow_speed = 5.0`. Orthographic projection uses `glm::ortho` with Y-flip (top > bottom). Camera shake applies random offsets scaled by a linear fade-out.

2.4 Input System

`input/input.h` maps SDL scancodes to an `Action` enum (13 actions: MoveUp/Down/Left/Right, Interact, Cancel, Pause, ZoomIn/Out, CameraUp/Down/Left/Right). Tracks three states per action: `pressed` (single frame), `held`, `released` (single frame). Call `begin_frame()` to clear transient flags.

2.5 Asset Manager

`core/asset_manager.h/cpp` provides shader caching and hot-reload:

- `load_shader(key, path, stage, ...)` — reads SPIR-V binary, caches by key
- `register_pipeline(key, vert_key, frag_key)` — tracks vert/frag dependency
- `check_for_updates()` — polls file `stat()` for mtime changes each frame
- On change: sets `dirty=true` on shader, `needs_rebuild=true` on dependent pipelines

2.6 Task System

`core/task_system.h/cpp` implements a thread pool using `std::mutex + std::condition_variable`. Workers wait on the CV, pick tasks from a deque, and decrement an `atomic<int> active_count`. `is_idle()` returns true when the queue is empty and active count is zero.

2.7 Watchdog

`core/watchdog.h/cpp` spawns a background thread that reads `/proc/self/status` for `VmRSS` every 500 ms. Default limit: 75% of total RAM (fallback 2 GB). On breach, sets `g_emergency_shutdown = true` and calls `std::quick_exit(1)` after a 500 ms grace period.

2.8 glTF Loader

`core/gltf_loader.h/cpp` loads static meshes via cgltf and textures via stb_image.

Table 3: GltfVertex layout (48 bytes)

Field	Type	Bytes	Offset
position	vec3	12	0
normal	vec3	12	12
texcoord	vec2	8	24
tangent	vec4	16	32

cgltf functions used: `cgltf_parse_file()`, `cgltf_load_buffers()`, `cgltf_validate()`, `cgltf_accessor_read_float()`, `cgltf_accessor_read_uint()`.

2.9 ImGui Integration

Custom SDL3-GPU backends (submodule in `imgui/`). Six-function API: `ui_init`, `ui_shutdown`, `ui_process_event`, `ui_begin_frame`, `ui_end_frame`, `ui_draw`. The tool window renders ImGui; the game window does not.

2.10 IPC Agent Server

`ipc/agent_server.h` provides a Unix domain socket with JSON-line protocol. Commands: `get_state`, `quit`, `capture_frame`, `send_input`. Non-blocking accept/read via `poll()` called each frame.

2.11 Background Renderer

`render/background.h/cpp` draws a full-screen triangle (3 vertices, no VBO) with a procedural starfield fragment shader. Pushes a 16-byte uniform (time, cam_x, cam_y, pad). Depth test and write are disabled—this is drawn first as the background layer.

2.12 Frame Capture

`gpu/frame_capture.h/cpp` copies the swapchain to a staging texture, downloads to a transfer buffer, fence-syncs, maps, and writes PNG via `stbi_write_png()`.

3 Terrain Pipeline

The terrain system in `src/game/terrain/` is the core of the project. It transforms noise parameters into GPU-ready hex-column meshes through six pipeline stages.

3.1 Step 1: Noise Generation

`noise.h/cpp` and `noise_layers.cpp` generate three noise layers using FastNoiseLite.

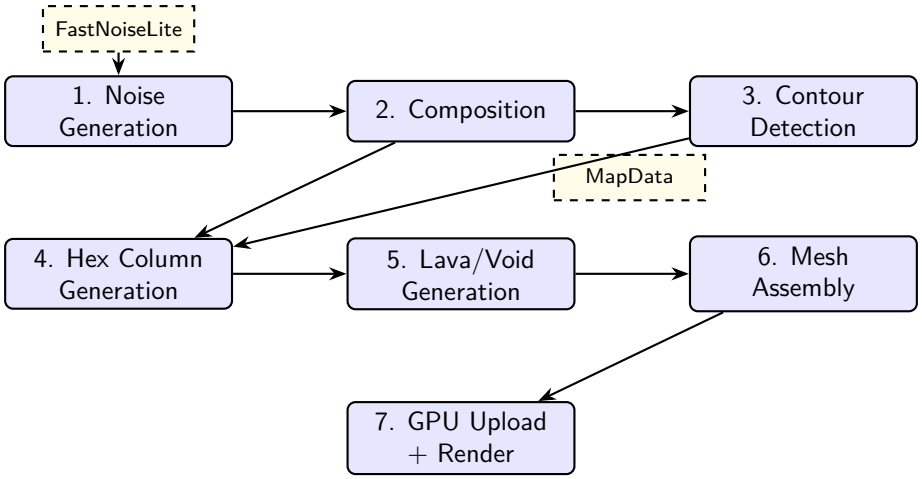

Figure 2: Terrain pipeline data flow

3.1.1 Elevation Layer

Listing 1: NoiseParams defaults

```
struct NoiseParams {
  float frequency = 0.003f;
  int   octaves = 6;
  float lacunarity = 2.0f;
  float gain = 0.5f;
  int   seed = 1337;
  int   terrace_levels = 8;
  int   min_region_size = 153;
};
```

Algorithm — Multi-octave OpenSimplex2 with gradient-based erosion:

Listing 2: Gradient erosion pseudocode

```
for each octave i:
  octave[x] = noise.GetNoise(x * map_scale, y * map_scale)
  grad_mag = sqrt(gradient_x[x]^2 + gradient_y[x]^2)
  erosion_factor = 1.0 / (1.0 + grad_mag * GRADIENT_SCALE)  //
     GRADIENT_SCALE = 2.0
  out[x] += octave[x] * amplitude * erosion_factor
  // Update gradient accumulator
  dx = (octave[x+1] - octave[x-1]) * 0.5
  gradient_x[x] += dx * amplitude * erosion_factor * frequency
  amplitude *= gain;  frequency *= lacunarity
```

Post-processing: normalize to $[0, 1]$, apply `biased_smoothstep` (blend of Hermite $3t^2 - 2t^3$ and quadratic ease-out $1 - (1-t)^2$, controlled by `scurve_bias`), then terrace: $\lfloor v \cdot L \rfloor / L$.

3.1.2 River Mask

Uses `FractalType_Ridged` with OpenSimplex2 (frequency 0.008, 4 octaves). The ridged fractal creates natural river-vein patterns. Output is min-max normalized across the entire map.

3.1.3 Worley (Cellular) Layer

Listing 3: WorleyParams defaults

```
struct WorleyParams {
  float frequency = 0.015f;
  float jitter = 1.0f;
  float warp_amp = 40.0f;        // domain warp amplitude
  float warp_frequency = 0.003f;
  int   warp_octaves = 3;
};
```

Three simultaneous outputs from one cellular noise evaluation:

1. **Distance** (`EuclideanSq`, `Distance`) — distance to nearest cell center
2. **Edge** (`Distance2Sub`) — difference between 1st and 2nd nearest distances
3. **Cell Value** (`CellValue`) — per-cell random ID

Domain warp: A separate FastNoiseLite instance (`seed + 31337`) with `DomainWarpType_OpenSimplex2` and `FractalType_DomainWarpProgressive` distorts the input coordinates before Worley evaluation, creating organic Voronoi boundary distortion.

How to Recreate

FastNoiseLite key calls: `SetNoiseType()`, `SetFrequency()`, `SetFractalType()`, `SetFractalOctaves/Lacunarity/Gain()`, `SetCellularDistanceFunction()`, `SetCellularReturnType()`, `SetCellularJitter()`, `SetDomainWarpType()`, `SetDomainWarpAmp()`, `DomainWarp(wx, wy)`, `GetNoise(x, y)`.

3.2 Step 2: Composition

`noise_composer.cpp` blends the three noise layers into the final terrain.

NoiseCache: A 3-slot cache using FNV-1a hashing of parameter structs. Avoids regenerating noise layers when only composition parameters change during ImGui tweaking.

Pipeline:

1. Copy elevation → `final_elevation`
2. Terrace: `basalt_height[i] = floor(elev[i] * levels) / levels`
3. `cleanup_small_regions()`: flood-fill regions of matching `basalt_height` (± 0.01 threshold); if region $<$ `min_region_size`, replace with 3×3 neighbor average

3.3 Step 3: Contour Detection

`contour.cpp` extracts elevation contour lines and detects plateaus.

3.3.1 Marching Squares

Listing 4: Marching squares pseudocode

```
for each cell (x, y), for each level L from interval/2 to 1.0:
  config = 4-bit mask of corners >= L
  if config in {0, 15}: skip (all above or below)
  // 2-point cases: interpolate crossing position on edges
  // 4-point cases (saddle): check cell center height to resolve
  emit Line{x1, y1, x2, y2, elevation=L}
```

Hard cap: `MAX_CONTOUR_LINES = 500,000`. Post-processing: `simplify_contours()` removes segments with squared length $< \varepsilon^2$.

3.3.2 Plateau Detection

BFS flood-fill on `band_map` (discrete elevation bands). Each plateau stores: pixel list, center of mass, bounding box, average height. Filter: only plateaus with > 50 pixels are retained.

Listing 5: Plateau structure

```
struct Plateau {
  float height;
  std::vector<int> pixels;
  float center_x, center_y;
  float min_x, max_x, min_y, max_y;
};
```

3.4 Step 4: Hex Column Generation

3.4.1 Axial Hex Coordinates

`hex.h/cpp` implements pointy-top hexagonal grids with axial coordinates (q, r).

$$\text{hex\_to\_pixel:} \quad x = s \cdot 1.5 \cdot q, \quad y = s \cdot \sqrt{3} \cdot (r + 0.5q) \tag{7}$$

$$\text{pixel\_to\_hex:} \quad q = \tfrac{2}{3} \cdot x/s, \quad r = (-\tfrac{1}{3} \cdot x + \tfrac{\sqrt{3}}{3} \cdot y)/s \tag{8}$$

Pixel-to-hex uses **cubic rounding**: convert to cube coordinates $(q, r, s = -q - r)$, round each, then fix the axis with the largest rounding error to maintain the $q + r + s = 0$ constraint.

Hex corners: 6 vertices at angles $i \cdot \pi/3$ for $i \in [0, 6)$, offset from hex center by `hex_size`. Point-in-hex test uses 6 half-plane cross-product checks.

3.4.2 Column Generation (`basalt.cpp`)

Two generation strategies exist:

V1 (Plateau-based): For each plateau ($\geq$300 pixels, aspect ratio $\leq$3.0), find center hex, BFS-expand outward. Each candidate hex is tested by checking a 3×3 pixel bounding box against `terrain_map`. Height variation: ± 0.025 via `hash(q,r)`.

V2 (Worley-based): Iterate all hexes in map bounds. Per hex: jitter position via hash, check `worley_cell_value` against `density_threshold` (default 0.2), skip if in `liquid_mask`. Height from base elevation plus cell-value jitter.

Visible edges: `compute_visible_edges()` checks each of 6 hex neighbors (offset table: $(1, 0), (0, 1), (-1, 1), (-1$ An edge is visible if the neighbor is absent or lower by >0.01.

3.5 Step 5: Lava/Void Generation

`lava.cpp` flood-fills non-basalt regions to create lava and void bodies.

Budget caps:

- `MAX_COMPONENT_PIXELS = 50,000` per body
- `TOTAL_PIXEL_BUDGET = 200,000` global

- Minimum component size: 50 pixels

Each component is randomly classified as void (30% chance) or lava (70%). Lava bodies get a mesh: a sparse grid (2.0 unit spacing) with 2 triangles per quad cell. Grid size capped at `MAX_LAVA_GRID_CELLS = 40,000`. Each body gets a random time offset for wave animation phase: offset = (hash(idx) mod 1000)/1000 · 2π.

3.6 Step 6: Mesh Assembly

`terrain_mesh.cpp` converts terrain data into GPU-ready vertex/index buffers.

Table 4: Vertex format layouts

Struct	Fields	Size	Usage
BasaltVertex	pos(3), color(3), sheen(1), normal(3)	48 B	Hex columns
GpuLavaVertex	pos(3), time_offset(1)	16 B	Lava surfaces
ContourVertex	pos(3)	12 B	Contour lines

Two-layer basalt mesh:

- **Layer 0 (sides):** Per visible edge, 4 vertices (top-near, top-far, bottom-far, bottom-near) and 2 triangles. Normal computed as 2D perpendicular to the edge direction. Side sheen = sheen × 0.4.
- **Layer 1 (tops):** 6 vertices per hexagon + 4 fan triangles from vertex 0. Normal = $(0, 0, 1)$.

Color is generated by `organic_color()`: palette interpolation based on elevation with per-hex hash noise for variation.

3.7 Step 7: GPU Upload & Rendering

`terrain_renderer.cpp` manages five GPU pipelines and the draw order.

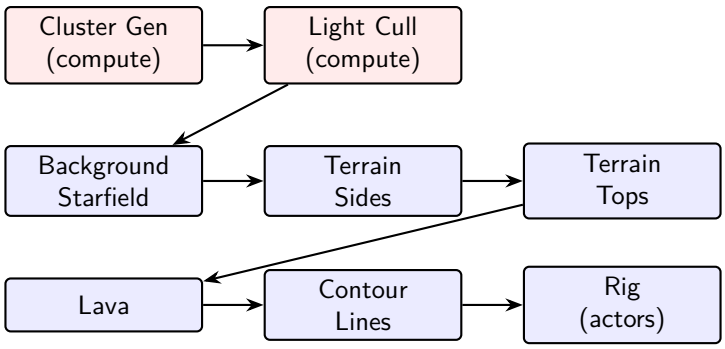

Figure 3: Render pass execution order

Clustered light culling (compute pass before rendering):

- `generate_clusters.comp`: 16 × 9 tile grid × 24 depth slices = 3456 clusters. Computes view-space AABB per cluster.
- `light_culling.comp`: sphere–AABB intersection test per light per cluster. Outputs light grid + global index list (max 65536 indices, max 1024 lights).
- Cluster index: `tileX` + `tileY` × `gridX` + `slice` × `gridX` × `gridY`

SceneUniforms (std140, pushed per draw call): view/projection matrices, time, contour opacity, lava color, star light (RGB + intensity), directional light (direction + color + ambient), grid parameters for clustering, near/far planes, light count.

4 Shader Architecture

All shaders are GLSL 4.5, compiled to SPIR-V. Three shared includes define common interfaces.

4.1 Shared Includes

`coord.glsl` — `SceneUniforms` UBO (set=1, binding=0) and the `cluster_index()` function that maps fragment position to a 3D cluster cell.

`lighting_common.glsl` — `PointLight` (32 B: pos+radius, color+intensity), `ClusterAABB` (min/max vec4), `LightGridEntry` (offset + count).

`pbr_common.glsl` — Cook-Torrance BRDF:

$$D_{\text{GGX}}(h) = \frac{\alpha^2}{\pi((\mathbf{n}\cdot\mathbf{h})^2(\alpha^2-1)+1)^2} \tag{9}$$

$$G_{\text{Schlick}}(v) = \frac{\mathbf{n}\cdot\mathbf{v}}{(\mathbf{n}\cdot\mathbf{v})(1-k)+k}, \quad k = \frac{(\alpha+1)^2}{8} \tag{10}$$

$$F_{\text{Schlick}}(\theta) = F_0 + (1-F_0)(1-\cos\theta)^5, \quad F_0 = \text{mix}(0.04, \text{albedo}, \text{metallic}) \tag{11}$$

4.2 Terrain Shaders

Vertex: Transforms position by view × projection. Passes color, normal, world position, sheen.

Fragment: Lambertian diffuse from directional light + star ambient + clustered point lights. Applies a hex dither function for subtle per-cell color variation:

```
// Hash hex cell center for dither
uint h = uint(iq) * 374761393u ^ uint(ir) * 668265263u;
float dither = (float(h & 0xFFu) / 255.0 - 0.5) * 0.25;
```

4.3 Lava Shaders

Vertex: Triple sine-wave deformation for animated surface:

```
float t = time + time_offset;
float wave = sin(pos.x * 0.3 + t) * 0.02
           + sin(pos.y * 0.21 + t * 1.3) * 0.015
           + sin((pos.x + pos.y) * 0.15 + t * 0.8) * 0.01;
pos.z += wave;
```

Fragment: Depth-based darkening: `depth_factor = clamp(0.9 + wave * 2.0, 0.8, 1.1)`.

4.4 Background Shaders

Full-screen triangle (vertex ID only, no VBO). Fragment generates a procedural starfield:

- 3 layers with cell sizes 120, 55, 28 pixels and spawn chances 0.45, 0.38, 0.30
- Per-star: hash-based position + twinkle (frequency 0.5–2.5 Hz, random phase)
- Core radius 0.3–0.7 px, glow = 3.5× core radius

- Radiance: e^{-d^2/r^2}, luminance 0.25–1.0
- Parallax scroll at 0.18× camera speed

4.5 Contour Shaders

Simple pass-through vertex shader. Fragment outputs a fixed color with configurable alpha. Blending: src-alpha / one-minus-src-alpha. No depth test or write—contours overlay everything.

4.6 Compute Shaders

Both use `local_size = (16, 9, 1)` matching the tile grid dimensions.

Cluster generation: Converts each tile + depth slice to a view-space AABB using inverse projection. Stores min/max bounds per cluster.

Light culling: For each cluster, tests all lights for sphere–AABB intersection. Writes matching light indices into a global index buffer with an atomic counter.

4.7 Instanced Terrain

SSBO at set=0 contains `ColumnInstance` (80 B: mat4 model + vec4 color). Vertex shader reads instance data by `gl_InstanceIndex`. Normal transform uses the inverse-transpose of the model matrix upper-left 3×3.

4.8 PBR Static

Drop-in replacement for the terrain pipeline. Uses Cook-Torrance BRDF from `pbr_common.glsl` with clustered lighting fallback (iterates all lights if cluster lookup fails).

5 Procedural Animation System (Rig System)

The rig system provides fully procedural character animation—no pre-baked animations, no motion capture. Everything is computed per-frame from physics and locomotion state.

5.1 Skeleton

`rig.h` defines a 32-joint skeleton:

Table 5: Joint hierarchy (32 joints, `Joint::COUNT = 32`)

Group	Joints
Core spine (8)	ROOT, HIPS, SPINE_01, SPINE_02, CHEST, NECK, HEAD, HEAD_END
Left arm (4)	L_CLAVICLE, L_UPPER_ARM, L_LOWER_ARM, L_HAND
Right arm (4)	R_CLAVICLE, R_UPPER_ARM, R_LOWER_ARM, R_HAND
Left leg (4)	L_UPPER_LEG, L_LOWER_LEG, L_FOOT, L_TOE
Right leg (4)	R_UPPER_LEG, R_LOWER_LEG, R_FOOT, R_TOE
IK virtual (8)	POLE_KNEE_L/R, POLE_ELBOW_L/R, IK_FOOT_L/R, IK_HAND_L/R

ActorConfig defines anthropomorphic proportions (all scaled by 0.85× reference):

- Legs: `leg_len=0.366`, `shin_len=0.332`
- Torso: `torso_len=0.520`, `neck_len=0.104`
- Arms: `arm_len=0.270`, `forearm_len=0.230`
- Widths: `hip_width=0.25`, `shoulder_width=0.35`
- `ISO_VERT_SCALE = 0.8165` (isometric Z compression)

`RigPose` holds world-space `vec3 joints[32]`. Derived joints (ROOT, SPINE_02, HEAD_END, clavicles, toes, pole targets, IK goals) are computed in SkeletonFinalise.

5.2 ECS System Pipeline

Nine systems registered as `flecs::PostUpdate`, executed in strict order.

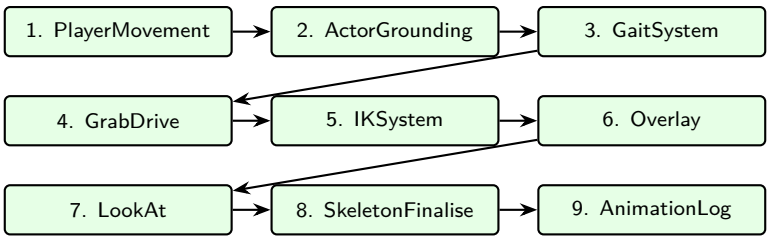

Figure 4: Animation ECS system execution order

5.2.1 1. PlayerMovementSystem

Maps WASD input to isometric movement axes (45° rotation: $\cos 45° = \sin 45° = 0.707$). Velocity smoothed via `SmoothDamp` with `smooth_time=0.1s`. Facing updated via `atan2` when speed > 0.001. Turn detection uses hysteresis: enter large-turn at $> 60°$, exit at $< 20°$. Visual facing has adaptive smooth time: 0.15 s (small turns) to 0.50 s (full reversal). Chest facing lags with 0.22 s (moving) or 0.06 s (idle).

5.2.2 2. ActorGroundingSystem

Snaps actor Z to terrain height via bilinear sampling of `final_elevation`.

5.2.3 3. GaitSystem — Distance-Based Procedural Stepping

The gait system is the most complex animation subsystem. It maintains a one-foot-planted invariant: exactly one foot is always grounded while the other may be stepping.

Key parameters: `stride_len=0.85`, `step_height=0.16`, `step_duration=0.25s`, `SWING_RATE=2.7 rad/s`.

Phase accumulation: `phase += speed * dt * SWING_RATE`

Unified step basis: The step direction is derived from a blend of velocity direction and visual facing (max 70% blend during turns). Both `step_fwd` and `step_rght` are computed from this unified basis, preventing asymmetric reach during turns.

Step triggering: A step is triggered when the planted foot's distance from the hip socket exceeds `trigger_dist = half_stride * (0.25 + 0.75 * speed_factor)`. Half-space constraint forces corrective steps when a foot crosses the center line.

Foot placement (velocity-predicted):

$$\text{target\_off} = (\text{half\_stride} + \text{speed} \cdot \text{duration} \cdot 0.75) \cdot \text{speed\_factor}$$

Foot animation:

- XY: smootherstep $t^3(10t^2 - 15t + 6)$ interpolation
- Z: $t^2(3 - 2t) + \sin(\pi t) \cdot \text{step\_height}$ (parabolic arc)

Turn-aware sequencing: The trailing foot (relative to turn direction) steps first when `turn_urgency` $>$ `0.4`. Starvation guard resets the queue when urgency drops below 0.4.

5.2.4 4. GrabDriveSystem

Maps `GrabState` to `ArmIKGoal` targets. Activates `HeavyCarry` overlay and reduces stride when carrying objects.

5.2.5 5. IKSystem — Two-Bone Analytical IK

Spine FK builds the chain bottom-up: HIPS → SPINE_01 → CHEST → NECK → HEAD, using torso segment lengths.

Arm swing (FK pendulum): Swing amplitude = $\min(1, \text{speed}/\text{move\_speed}) \times 30°$. Left and right arms are anti-phase: $l = \sin(\phi + \pi)$, $r = \sin(\phi)$. Joint delay chain for successive breaking: shoulder 0.02 s, elbow 0.04 s, wrist 0.06 s smooth-damp times. Rotation applied via Rodriguez formula around the chest-right axis.

Two-bone IK solver:

Listing 6: Two-bone analytical IK

```
void solve_two_bone(vec3 H, vec3 target, float a, float b,
                    vec3 pole, vec3 fallback_perp,
                    vec3 &out_mid, vec3 &out_end) {
  float D = clamp(length(target - H), |a-b|+0.001, a+b-0.001);
  float cos_alpha = (a*a + D*D - b*b) / (2*a*D);  // law of cosines
  vec3 dir = normalize(target - H);
  vec3 perp = normalize(pole_projected);  // or fallback_perp
  out_mid = H + (dir * cos_alpha + perp * sin_alpha) * a;
  out_end = target;
}
```

Per-leg fallback perpendiculars: left knee gets (−rght), right gets (+rght), preventing shared fallback from pushing both knees the same direction.

5.2.6 6. OverlaySystem — Additive Overlays

Three overlay types applied additively to the pose:

- **Limp:** Asymmetric left hip drop: $\sin(\phi) \times 0.04 \times \text{intensity}$
- **Fatigue:** Increased sway + forward lean (0.03 rad) + slow breathing
- **HeavyCarry:** Forward lean (0.05 rad) + pulled-down hands ($-0.06 \times$ intensity)

5.2.7 7. LookAtSystem — Head Tracking

Decomposes target direction into yaw/pitch relative to facing. Clamped: yaw $\in [-70°, 70°]$, pitch $\in [-30°, 30°]$. SmoothDamp with 0.08 s. Distribution: HEAD 60%, NECK 30%, CHEST 10%.

5.2.8 8. SkeletonFinaliseSystem

The most complex system, performing multiple physics-driven corrections:

Slope hip tilt: From foot height difference, clamped $\pm 8°$. Applied to hip socket Z as $\sin(\text{tilt}) \times$ hip_width.

CoM hip sway: Support balance $\in [-1, +1]$ smoothed with 0.08 s. Lateral displacement = balance $\times$ 0.04.

Hip counter-animation:

$$\text{hip\_roll} = \sin(\phi) \times 0.06 \times \text{walk\_blend} \tag{12}$$

$$\text{hip\_bob} = |\sin(\phi)| \times 0.018 \times \text{walk\_blend} \times (1 - \text{turn\_urgency} \times 0.7) \tag{13}$$

$$\text{hip\_dip} = 4p(1-p) \times (\text{leg\_len} + \text{shin\_len}) \times 0.07 \quad \text{(parabolic during swing)} \tag{14}$$

Acceleration-driven torso lean: lean factor = $\min(|\mathbf{a}| \times 0.015, 8°)$, with additional 2.5° forward lean when moving. Applied via successive breaking to chest (0.05 s), neck (0.08 s), head (0.12 s).

Idle micro-motion: Breathing at 0.6 Hz (amplitude 0.008), lateral sway at 0.15 Hz (amplitude 0.01), weight shift at 0.3 Hz (hip shift 0.04, dip 0.012). All scaled by idle_blend = $1 - \min(1, \text{speed}/0.2)$.

5.2.9 9. AnimationLogSystem

Outputs JSONL telemetry per frame: transform, velocity, gait phase, all 32 joint positions, arm swing diagnostics, spine offsets, grounding state.

5.3 SmoothDamp — Critically Damped Spring

Listing 7: SmoothDamp implementation (Unity-compatible)

```
float smooth_damp(float current, float target, float *velocity,
                  float smooth_time, float dt) {
  float omega = 2.0f / smooth_time;
  float x     = omega * dt;
  float exp_f = 1.0f / (1 + x + 0.48f*x*x + 0.235f*x*x*x);
  float delta = current - target;
  float temp  = (*velocity + omega * delta) * dt;
  *velocity   = (*velocity - omega * temp) * exp_f;
  return target + (delta + temp) * exp_f;
}
```

The angle-aware variant wraps `delta` to $[-\pi, \pi]$ before smoothing.

6 Rig Renderer

`rig_renderer.cpp` generates procedural bone-cage meshes entirely at runtime—no pre-baked geometry.

Vertex budget: 65,536 vertices per frame (2.5 MB persistent transfer buffer).

Emit functions:

- `emit_cylinder()` — n-sided cylinder with computed normals
- `emit_bone_oct()` — tapered octahedron (4 sides, 20% equator), classic rig viz

- `emit_sphere()` — UV sphere (sectors × rings)
- `emit_box()` — 6-face solid cube
- `emit_diamond()` — 6-vertex octahedron (8 triangles)
- `emit_flat_circle()` — fan-triangulated disc on XY plane

Isometric compensation adjusts bone rendering for the isometric view:

$$\text{width\_scale} = 1.0 + \sqrt{|\mathbf{d} \cdot \hat{z}|} \times 2.0 \quad \in [1, 3] \tag{15}$$

$$\text{equator\_t} = 0.2 + |\mathbf{d} \cdot \hat{z}| \times 0.15 \tag{16}$$

$$\text{radial\_z\_comp} = 0.18 + |\mathbf{d} \cdot \hat{z}| \times 0.82 \tag{17}$$

Where $\mathbf{d}$ is the bone direction. Bones pointing more vertically get wider cross-sections to remain visible under isometric foreshortening.

Body part rendering: Spine (4 segments, 6-sided, radii 1.3–1.4×), head (2 segments, taper 0.4–0.3×), legs (4 segments each, 1.4–0.3×), arms (4 segments each, 1.5–0.75×).

Debug shapes: IK goals as green boxes (0.04 half-size), pole targets as orange diamonds (0.04 radius), pole lines as dashed orange cylinders.

7 glTF Asset Pipeline

7.1 Loading Pipeline

`load_gltf(path)` uses cgltf for parsing and stb_image for textures:

1. `cgltf_parse_file()` — parse the .glb/.gltf file
2. `cgltf_load_buffers()` — load binary buffer data
3. `cgltf_validate()` — validate the asset
4. Extract mesh attributes via `cgltf_accessor_read_float/uint()`
5. Decode embedded textures via `stbi_load_from_memory()`

Returns `GltfAsset` containing vectors of `GltfMeshData` (vertices + indices) and `GltfTextureData` (RGBA pixels, sRGB flag).

7.2 Instanced Terrain Rendering

`instanced_terrain.h/cpp` renders glTF column meshes via GPU instancing:

Listing 8: ColumnInstance layout (80 bytes)

```
struct ColumnInstance {
  glm::mat4 model;      // 64B: translate(q,r) * scale(1,1,height)
  float color_r, color_g, color_b, color_a;  // 16B
};
```

Instances stored in an SSBO (`GRAPHICS_STORAGE_READ`). The vertex shader reads instance data by `gl_InstanceIndex`. Toggle via `terrain_renderer.use_instanced` (ImGui checkbox).

8 ECS Architecture

Delve uses Flecs v4.0.5 for entity-component-system organization. All animation systems are registered as `flecs::PostUpdate` phase.

8.1 Component Catalog

Table 6: ECS components on the player entity

Component	Purpose
`ActorTag`	Tag component for entity queries
`Transform`	Position (x, y, z) + facing angle
`Velocity`	Movement velocity (x, y, z)
`ActorConfig`	Skeletal proportions (bone lengths, radii)
`ProceduralGait`	Gait phase, stride length, step height/duration
`LegState`	Per-leg foot positions, stepping flags, plant positions
`RigPose`	32 world-space joint positions
`RigState`	Animation smoothing state (velocity, lean, sway, etc.)
`LookAtTarget`	Head tracking target position
`ArmIKGoal`	Arm IK target positions + blend weights
`AnimationOverlay`	Additive overlay type (None/Limp/Fatigue/HeavyCarry)
`GrabState`	Object grab state for arm IK

8.2 System Registration Pattern

Listing 9: Flecs system registration

```
ecs.system<Transform, Velocity, ActorConfig, ProceduralGait,
           LegState, RigPose, RigState>("GaitSystem")
   .kind(flecs::PostUpdate)
   .run([](flecs::iter &it) {
     while (it.next()) { /* system logic */ }
   });
```

All 9 animation systems are registered in `register_rig_systems()` with explicit ordering via Flecs phase dependencies.

9 Testing Framework

9.1 Test Harness

`src/test/test_harness.h` provides a lightweight custom test framework.

Registration: `DELVE_TEST(name) { ... return true; }` — creates a static function and registers it with `TestRegistry` at compile time.

Assertion macros:

Output: JSON to stdout: `{"tests":[...], "passed":N, "failed":N, "total":N}`.

Headless testing: SDL3 is stubbed out via headers in `src/test/sdl_override/` that intercept includes. The `DELVE_HEADLESS=1` compile definition disables GPU code paths.

Macro	Condition
`EXPECT_TRUE(e)`	`e` is truthy
`EXPECT_FALSE(e)`	`e` is falsy
`EXPECT_NEAR(a,b,e)`	$\|a - b\| \leq \varepsilon$
`EXPECT_RANGE(v,l,h)`	$l \leq v \leq h$
`EXPECT_GT/LT/GE/EQ`	Relational comparisons

9.2 Metric Suites

Terrain metrics (`terrain_metrics.h`):

- `elevation_stats()` — min, max, mean, stddev
- `water_coverage_percent(data, threshold=0.1)`
- `plateau_count(band_map)` — max band ID
- `lava/void/basalt_coverage_ratio()` — percentage of terrain_map

Geometry metrics (`geometry_metrics.h`):

- `mesh_degenerate_triangles()` — cross product magnitude $< 10^{-12}$
- `normal_validity()` — percentage with $|\|\mathbf{n}\| - 1| < 0.01$
- `hex_roundtrip_accuracy()` — tests hex→pixel→hex roundtrip

Animation metrics (`animation_metrics.h`):

- `pose_symmetry_score()` — compares 6 L/R joint pairs, returns $1/(1 + \text{diff})$
- `foot_contact_velocity()` — lower is better, 0 = perfect plant
- `arm_phase_opposition()` — 1.0 if arms counter-phase to legs
- `lean_acceleration_correlation()` — validates physics-driven lean

9.3 Test Files

9 test source files covering: noise generation, hex coordinates, animation, color palettes, full terrain pipeline, mesh integrity, coordinate system roundtrips, skeletal animation, and skinned characters.

10 Library Reference

Library	Key Functions Used	Purpose in Delve
SDL3	`SDL_CreateGPUDevice`, `SDL_AcquireGPUSwapchainTexture`, `SDL_CreateGPUGraphicsPipeline`, `SDL_CreateGPUComputePipeline`, `SDL_BeginGPURenderPass`, `SDL_DrawGPUPrimitives`, `SDL_DispatchGPUCompute`, `SDL_PushGPUVertexUniformData`, `SDL_BindGPUVertexStorageBuffers`, `SDL_CreateWindow`, `SDL_PollEvent`	Windowing, GPU device management, render pass control, draw calls, compute dispatch, uniform upload, event polling
Flecs	`world::entity()`, `world::system()`, `entity::set<T>()`, `entity::add<T>()`, `system::kind()`, `system::run()`, `iter::field<T>()`, `world::progress()`	Entity creation, component attachment, system registration with phase ordering, per-frame system execution
GLM	`glm::ortho`, `glm::normalize`, `glm::cross`, `glm::dot`, `glm::length`, `glm::mix`, `glm::clamp`, `glm::translate`, `glm::scale`, `glm::inverse`, `glm::transpose`, `glm::mat4`, `glm::vec3/vec4`	Orthographic projection, vector math, matrix transforms, isometric view construction, bone basis computation
FastNoiseLite	`SetNoiseType(OpenSimplex2)`, `SetFrequency`, `SetFractalType(Ridged/None)`, `SetFractalOctaves/Lacunarity/Gain`, `SetCellularDistanceFunction(EuclideanSq)`, `SetCellularReturnType(Distance/Distance2Sub/CellValue)`, `SetCellularJitter`, `SetDomainWarpType(OpenSimplex2)`, `SetDomainWarpAmp`, `DomainWarp(x,y)`, `GetNoise(x,y)`	Elevation heightmap (multi-octave with erosion), ridged river mask, Worley cellular (3 outputs), domain warp for coastlines
ImGui	`ImGui::Begin/End`, `SliderFloat/Int`, `ColorEdit3`, `Checkbox`, `Text`, `Separator`, `TreeNode`, `NewFrame`, `Render`, `GetDrawData`	Real-time parameter editor for noise, composition, display, and rendering settings

Library	Key Functions Used	Purpose in Delve
cgltf	`cgltf_parse_file`, `cgltf_load_buffers`, `cgltf_validate`, `cgltf_accessor_read_float`, `cgltf_accessor_read_uint`, `cgltf_free`	glTF 2.0 parsing for static mesh loading (basalt column asset)
stb_image	`stbi_load_from_memory`, `stbi_image_free`	Decode embedded glTF textures (base color, normal, ORM maps)
stb_image_write	`stbi_write_png`	Frame capture: swapchain → PNG file
nlohmann/json	`json::parse`, `json::dump`, `operator[]`, `value<T>()`, `contains()`	Config save/load (`config.json`), test JSON output, IPC message parsing, animation telemetry

11 Entry Point & Runtime Flow

`src/game/main.cpp` installs signal handlers (SIGTERM, SIGINT → `g_emergency_shutdown`), parses command-line arguments via `parse_agent_args()`, creates a `TopoGame` instance, and calls `game.run(agent_config)`.

`TopoGame` (subclass of `Application`) overrides the lifecycle hooks to:

1. **on_init:** Load `config.json`, generate initial terrain, upload meshes to GPU, create player entity with all 12 components, register all 9 animation systems
2. **on_render_tool:** Draw ImGui parameter editor (noise, composition, display settings)
3. **on_render_game:** Execute render pass order: compute (cluster gen → light cull) → background → terrain → lava → contours → rig
4. **on_event:** Forward to input system and ImGui
5. **on_cleanup:** Release GPU resources, save config

Runtime configuration is persisted in `config.json` at the project root.